

UNIVERSIDAD
DE GRANADA

Artificial Bee Colony

Práctica opcional

Pablo Linari Pérez

DNI: 75571079W
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del algoritmo 3
1.1 Funcionamiento del algoritmo 3
1.2 Parámetros del algoritmo 3
1.3 Ecuaciones 4
1.4 Etapas del algoritmo 4
1.4.1 Etapa 1: Inicialización 4
1.4.2 Etapa 2: Fase de abejas empleadas 4
1.4.3 Etapa 3: Fase de abejas observadoras 5
1.4.4 Etapa 4: Fase de abeja exploradora 5
1.5 Pseudocódigo 5
1.6 Adaptaciones al problema del portfolio 6
1.7 Mejoras del algoritmo 7
1.7.1 Hibridación con Local Search 7

1 Descripción del algoritmo

El algoritmo **Artificial Bee Colony** (ABC), propuesto por Dervis Karaboga en 2007, es una metaheurística poblacional inspirada en el comportamiento de búsqueda de alimento de las colonias de abejas. La idea del algoritmo es que un conjunto de abejas explora el espacio de búsqueda en el que hay una serie de fuentes de alimentos, intensificando la exploración alrededor de las fuentes más prometedoras y abandonando aquellas que se agotan.

En el contexto de este problema cada **fuente de alimento** representa una solución candidata, y la cantidad de néctar de una fuente se corresponde con el valor de fitness de esa solución. El objetivo del algoritmo es, por tanto, encontrar la fuente con mayor cantidad de néctar, es decir, la solución que maximiza la función objetivo.

==Explicación de los componentes del algoritmo

La colonia se divide en tres tipos de abejas, cada uno con un papel distinto dentro de la búsqueda:

- **Abejas obreras (worker bees)**: cada una está asociada a una fuente de alimento concreta y se encarga de explotarla buscando soluciones mejores en su vecindario.
- **Abejas observadoras (onlooker bees)**: esperan en la colmena y eligen qué fuentes explotar en función de la información que comparten las obreras. Las fuentes con más néctar tienen más probabilidad de ser elegidas, esto hace que se concentre la búsqueda en las funetes de alimento más prometedoras.
- **Abejas exploradoras (scout bees)**: cuando una fuente se agota se abandona, y la abeja obrera que trabajaba sobre esta se convierte en exploradora, buscando una fuente de alimento nueva.

El número de fuentes de alimento, según se indica en el artículo, es la mitad del tamaño total de la colonia, y coincide tanto con el número de abejas trabajadoras como con el de abejas observadoras ya que cada fuente tiene una trabajadora asociada.

Esta forma de inicializar la población es la propuesta en el artículo , no obstante veremos que se puede modificar este parámetro para cambiar el comportamiento del algoritmo aunque no sea el parámetro que más influya.

1.1 Funcionamiento del algoritmo

1.2 Parámetros del algoritmo

El comportamiento del algoritmo se controla mediante los siguientes parámetros:

- **colonySize**: tamaño total de la colonia. El número de fuentes de alimento se calcula como $SN = \frac{\text{colonySize}}{2}$. En la implementación se usa un valor de 50, por lo que $SN = 25$.
- **Limit** : número máximo de ciclos que una fuente puede permanecer sin mejorar antes de ser abandonada. Controla cuanto se pueden explotar las soluciones.
- **maxevals**: número máximo de evaluaciones de la función objetivo, que actúa como criterio de parada.

1.3 Ecuaciones

El algoritmo usa dos ecuaciones . La primera genera una solución candidata v_i a partir de una fuente existente x_i, perturbando una única componente j elegida al azar:

$$v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj})$$

donde $k \neq i$ es otra fuente vecina elegida aleatoriamente y ϕ_{ij} es un número aleatorio en el intervalo $[-1, 1]$. El resto de componentes de v_i se copian directamente de x_i.La nueva solución se acerca o se aleja de una fuente vecina en una sola dimensión.

Para entenderlo mejor , este movimiento en el problema del portfolio equivale a tomar una solución i , y sumar o resar a una empresa de dicha solución un porcentaje de la misma empresa pero de la otra solución escogida k.De esta forma nos acercaríamos o alejaríamos de la solución escogida en dicha componente.

La segunda ecuación calcula la probabilidad de que cada fuente sea seleccionada por una observadora, de forma proporcional a su néctar:

$$p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN} \text{fit}_n}$$

Cuanto mayor es el fitness de una fuente, mayor es su probabilidad de ser explotada por las observadoras.

1.4 Etapas del algoritmo

La ejecución se divide en 4 fases claras donde se identifica el comportamiento de cada abeja con su rol.

1.4.1 Etapa 1: Inicialización

Se generan SN fuentes de alimento de forma aleatoria, asegurando que cada una cumple las restricciones del problema (en el caso del portfolio se comprueban las restricciones necesarias). Se evalúa el fitness de cada una y se memoriza la mejor solución encontrada hasta el momento. Además, se inicializa a 0 un contador de *trials* por cada fuente, que registra cuántos ciclos lleva sin mejorar.

1.4.2 Etapa 2: Fase de abejas empleadas

Cada abeja empleada explota su propia fuente x_i. Para ello genera una solución candidata v_i aplicando la ecuación (2.2) y la evalúa. A continuación se aplica una selección greedy: si la candidata es mejor que la fuente original, la sustituye y el contador de *trials* de esa fuente se reinicia a 0, en caso contrario, la fuente se mantiene y su contador de *trials* se incrementa. Esta fase genera por cada fuente de alimento una posible solución mejor usando la ecuación previamente vista, si esta solución es mejor será la que se use como base para la explotación.

1.4.3 Etapa 3: Fase de abejas observadoras

Esta etapa consiste en el «baile de las obreras» donde cada obrera vuelve a la colmena e intentará convencer a las observadoras de elegir su fuente de alimento , las observadoras eligirán con mayor probabilidad la fuente que más nectar tenga.Los pasos que se siguen una vez la obrera vuelve a la colmena son los siguientes. Primero se calculan las probabilidades de selección de cada fuente con la segunda ecuación . Después, se reparten SN observadoras: cada una elige una fuente mediante selección por ruleta ,(las fuentes con más néctar tienen más probabilidad) y la explota igual que una empleada, usando la misma ecuación y aplicando de nuevo la selección greedy. Como las mejores fuentes tienden a ser elegidas varias veces, esta fase intensifica la búsqueda en las regiones más prometedoras del espacio. Es decir que podemos tener varias abejas observadoras en la misma fuente de alimento, esto ocurrirá sobre todo en las fuente más prometedoras donde como es lógico, interesa intensificar la búsqueda.

1.4.4 Etapa 4: Fase de abeja exploradora

Al final de cada ciclo se comprueba si alguna fuente se ha agotado. Se localiza la fuente con mayor número de *trials* acumulados y, si este supera el *Limit*, dicha fuente se abandona, se sustituye y la abeja exploradora buscaría una nueva, que a modo práctico consiste en crear una nueva solución aleatoria,se evalúa la nueva solución y su contador de *trials* vuelve a 0. Se permite como máximo una abeja exploradora por ciclo. Esta fase introduce diversificación y evita que el algoritmo quede estancado en óptimos locales.

En las tres fases se actualiza siempre la mejor solución global encontrada, que es la que finalmente devuelve el algoritmo.

1.5 Pseudocódigo

```
funcion ABC(problema, maxevals):
  SN = colonySize / 2
  # --- Inicialización ---
  para i en 0..SN:
    fuentes[i]   = problema.createSolution()
    fitness[i]   = problema.fitness(fuentes[i])
    trials[i]    = 0
    actualizar_mejor(fuentes[i], fitness[i])

  # --- Ciclos de búsqueda ---
  mientras evals < maxevals:

    # (a) Fase empleadas
    para i en 0..SN:
      candidata = produceCandidate(fuentes, i)   # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)
```

```
    # (b) Fase observadoras
    probs = computeProbabilities(fitness)         # ec. 2.1
    para t en 0..SN:
      i = rouletteSelect(probs)
      candidata = produceCandidate(fuentes, i)    # ec. 2.2
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)

    # (c) Fase exploradora (máx. una por ciclo)
    peor = indice con mayor trials
    si trials[peor] >= trialLimit:
      fuentes[peor] = problema.createSolution()
      trials[peor]  = 0
      actualizar_mejor(fuentes[peor])

  devolver mejor_solucion
```

1.6 Adaptaciones al problema del portfolio

La formulación original de ABC está pensada para la optimización de funciones numéricas sin restricciones, por lo que ha sido necesario realizar dos adaptaciones:

- **Reparación de soluciones**: la ecuación que se usa para explorar nuevas soluciones modifica una componente y, en general, rompe las restricciones del problema (los pesos deben sumar 1 y estar en $[\text{low}, \text{high}]$ o ser 0). Por ello, tras generar cada candidata se comprueba su validez y, si no es factible, se repara con `problem.fix()` antes de evaluarla.
- **Desplazamiento de las probabilidades**: la ecuación (2.1) asume cantidades de néctar no negativas, pero el fitness del portfolio puede ser negativo, de hecho en la práctica es siempre negativo. Para poder aplicar la ruleta, se desplazan todos los valores de fitness sumándoles una constante que deja el peor de ellos ligeramente por encima de 0, conservando las diferencias relativas entre fuentes.

Por lo que respecta a la implementación , la diferencia entre la implementación para el problema del portfolio y para el **cec2017** son las mencionadas previamente y el uso de las funciones de la clase **pPortfolio** , principalmente usadas para poder arreglar y generar soluciones con las restricciones del problema.

1.7 Mejoras del algoritmo

1.7.1 Hibridación con Local Search

Com primera opción de mejora se opta por hibridarlo con la Búsqueda Local, el enfoque que se usa puede ser de dos maneras distintas.

- Como primera prueba se ha usado la búsqueda local para intensificar la búsqueda sobre la mejor solución. Esta búsqueda local no se ejecuta sobre todas las posibles mejores soluciones, sino que se tiene un contador y cada